MeetingMind

Technical & NLP Architecture Report

An evidence-grounded meeting-intelligence pipeline: local NLP models, no paid APIs, every output traceable back to the transcript sentence it came from.

Generated 24 October 2026

React + TypeScript frontend • FastAPI + local NLP backend

Pretrained neural models	Extraction categories	Automated tests	Unseen-data micro-F1
3	8	127	0.76 vs 0.53 baseline

1. Abstract

Meetings produce decisions and commitments that are easy to lose in long transcripts. MeetingMind converts unstructured, multi-speaker transcripts into structured meeting intelligence using a **hybrid pipeline** — linguistic rules over a dependency parse, combined with sentence-embedding semantics and a dialogue-tuned abstractive summarizer, rather than a single large language model behind a paid API. Linguistic analysis (spaCy tokenisation, POS tagging, dependency parsing, NER) finds commitment, decision and risk cues. Sentence embeddings (`all-MiniLM-L6-v2`) provide semantic verification, topic segmentation, keyphrase re-ranking and de-duplication. A DistilBART model fine-tuned on dialogue (SAMSum) produces hierarchical abstractive summaries, which are then checked against the transcript for hallucinated names, numbers and claims. Relative deadlines (“next Friday”, “end of sprint”) are normalised against the meeting date with explicit, documented rules — never invented. On a held-out set of three transcripts written after development finished, the system reached a **micro-F1 of 0.76**, against **0.53** for a keyword baseline, with owners and deadlines **100% correct** on the action items it found.

Everything runs locally on CPU — no OpenAI/Anthropic/paid inference calls are made anywhere in the pipeline. The three neural models together total roughly **400 MB** of weights and load once per server process.

2. Problem Statement

Transcripts from Zoom, Teams or manual notes are long, informal and spread across speakers. “Can you send that by Friday?” only becomes an action item when you know who “you” is (often the next speaker who says “Sure”) and which Friday is meant. Generic summarizers skip these details or invent them. MeetingMind needed to:

- extract *who* will do *what* by *when*, without inventing owners or dates;
- separate real decisions from suggestions and hypotheticals;
- surface risks, blockers and questions nobody answered;
- show the exact evidence for every claim, so people can trust or correct it.

Scope

In scope: English transcripts as text; TXT / DOCX / text-based PDF / VTT / SRT upload; single-user local deployment; CPU-friendly models.

Out of scope: audio transcription (speech-to-text), OCR of scanned PDFs, accounts or multi-user collaboration, persistent storage, non-English languages.

3. System Architecture

MeetingMind has two parts. A **stateless FastAPI backend** runs the NLP pipeline and never writes a transcript to disk or a database. A **React single-page app** holds the report in memory, lets the user edit it, and asks the backend to render exports — so exported files always include the user's edits, since the export is generated from whatever the browser currently sends.

Request lifecycle

- **Upload / paste** — `POST /api/extract` pulls text out of TXT, DOCX, text-based PDF, VTT or SRT files.

- **Analyze** — `POST /api/analyze/stream` streams NDJSON progress events for 11 pipeline stages while a worker thread runs the pipeline (one analysis at a time; others receive a `queued` event).
- **Review & edit** — the Results page renders 12 views of the same report; action items, owners, deadlines and statuses are editable in place.
- **Export** — `POST /api/export/<format>` renders PDF, Markdown, CSV or JSON server-side from the (possibly edited) report the client sends back.

Backend modules

Module	Responsibility
app/main.py	App factory: CORS, body-size limits, a validation-error handler that never echoes request bodies, optional model preloading
app/config.py	Settings read from environment variables / .env (model names, device, upload limits)
app/nlp/models.py	ModelRegistry: lazy thread-safe loading, load timing, failure capture and retry per model
app/nlp/preprocessing.py	Normalisation, speaker/timestamp grammar, VTT/SRT parsing, utterance merging, spaCy sentence segmentation
app/nlp/common.py	Embedder (with hashed n-gram fallback), PrototypeScorer, NameResolver, evidence helpers
app/nlp/actions.py / deadlines.py / decisions.py / risks.py / questions.py	The five extractors — see Section 6
app/nlp/entities.py / keywords.py / topics.py	NER aggregation & speaker stats, keyphrases, topic segmentation
app/nlp/summarizer.py	Hierarchical abstractive summarization and grounding (hallucination) checks
app/nlp/next_steps.py	Rule-based recommendations linked to evidence
app/nlp/pipeline.py	Orchestration, stage progress events, validation, statistics, report assembly
app/services/file_extraction.py	TXT/DOCX/PDF/VTT/SRT extraction with temp-file cleanup and magic-byte checks
app/services/exports.py	JSON, CSV (formula-injection safe), Markdown, PDF (ReportLab) export rendering

Robustness & fallbacks

Failure	Behaviour
Summarizer fails to load/download	Falls back to an extractive TextRank summary; warning shown in the report
Embedding model fails	Hashed TF-IDF-style vectors; semantic checks disabled, rule-only matching; warning
spaCy model fails	Blank spaCy pipeline + sentencizer; no NER/parser, speaker labels still give people; warning
Client disconnects mid-analysis	Server finishes the current run and discards the result; nothing is stored

4. Technology Stack

Backend (Python 3.10+)

- **FastAPI** + **uvicorn** — async HTTP API and NDJSON progress streaming
- **Pydantic** — request/response schema validation (single source of truth, mirrored by the frontend's TypeScript types)
- **PyTorch** + **Hugging Face Transformers** — runs the DistilBART summarizer (CPU build is sufficient)
- **Sentence-Transformers** — loads and runs the MiniLM embedding model
- **spaCy** — tokenisation, POS tagging, dependency parsing, NER
- **YAKE** — unsupervised statistical keyword extraction (no pretrained weights)
- **dateparser** — parses explicit calendar dates found in text
- **scikit-learn** — only used for the hashed-vector fallback when the embedding model is unavailable
- **python-docx**, **pypdf**, **reportlab** — file text extraction and PDF export rendering

Frontend

- **React 19** + **TypeScript**, built with **Vite**
- **Tailwind CSS v4** — token-driven theming (see Section 10) with a dark and a light palette
- **React Router** — client-side routing for Dashboard / New Analysis / Processing / Results
- **Recharts** — the talk-share and keyword bar charts
- **lucide-react** — icon set
- **Vitest** + Testing Library — 15 component/integration tests

5. Pretrained Models

MeetingMind uses **three** pretrained neural models, all downloaded once from Hugging Face on first run and then loaded locally — no transcript text ever leaves the machine, and no paid inference API is called. A thread-safe `ModelRegistry` loads each model lazily and at most once per server process, caching it and recording load time, state (`not_loaded / loading / ready / failed`) and any error, all exposed through `/api/health`.

Model	Library	Role in the pipeline	Size / architecture
`en_core_web_sm`	spaCy	Sentence segmentation, POS tags, dependency parse, lemmas, noun chunks, named entities (PERSON, ORG, GPE, DATE, …)	~13 MB small English pipeline (CNN-based)
`sentence-transformers/ all-MiniLM-L6-v2`	Sentence-Transformers	Maps every sentence to a 384-dimensional vector; cosine similarity drives semantic classification, de-duplication, topic segmentation, keyword ranking and summary grounding (full detail in Section 7)	~90 MB, 6-layer MiniLM distilled from BERT
`philschmid/distilbart-cnn-12-6-samsum`	Transformers	Hierarchical abstractive summarization of the dialogue, followed by evidence-based grounding checks	DistilBART (12-layer encoder / 6-layer decoder, distilled from BART-large), fine-tuned on SAMSum dialogue-summary pairs; ~306M parameters

*Not a pretrained model: **YAKE** (unsupervised statistical keyphrase scoring — no training data or weights) and **dateparser** (rule/grammar-based date parsing). **scikit-learn** is present only to build a hashed bag-of-n-grams vector as a fallback if the embedding model cannot load.*

Why these three, and why not a single large language model

The project had to run **offline, on a laptop, with no paid APIs**. A single general-purpose LLM can write fluent summaries but cannot reliably extract structured fields (owner, deadline, status) on its own, and its reasoning can't be inspected or explained. The chosen approach instead combines:

- **Rules over a dependency parse** for commitment, decision and risk patterns — precise, fast, and fully explainable: every match can point to the exact grammatical pattern that fired.
- **Sentence embeddings** to add semantic generalisation on top of the rules, and to act as a *veto* — a rule match with a strongly negative similarity margin against positive examples is rejected even if the surface pattern matched.
- **One small, dialogue-tuned abstractive model** (DistilBART/SAMSum) for the parts genuine language generation is actually needed for — summarisation — with its output independently checked against the transcript rather than trusted.

Each decision taken by the pipeline can therefore be traced back to a rule, a similarity score, or a model output plus its grounding check — which matters because people act on this output.

Model selection notes

- **MiniLM-L6-v2** was chosen over larger sentence-embedding models because it is small (90 MB), fast enough to embed every sentence of a transcript on CPU in milliseconds, and scores strongly on semantic-textual-similarity benchmarks relative to its size.
- **DistilBART/SAMSum** was chosen over a plain news-summarization model (e.g. CNN/DailyMail-tuned BART) because dialogue-tuned models handle `Speaker: text` input far better. In a head-to-head comparison during development it was also more faithful than `knkarthick/MEETING_SUMMARY`, which stated a conditional postponement as settled fact.
- **en_core_web_sm** (the small spaCy English pipeline) was chosen over the larger `en_core_web_trf` transformer pipeline purely for CPU speed; the trade-off is that some organisation/product entities are mislabelled (documented in Section 11).

6. Embeddings, in Depth

Sentence embeddings are the single piece of machine learning that touches the most of the pipeline. `all-MiniLM-L6-v2` maps a sentence of any length to a fixed 384-dimensional vector such that the **cosine similarity** between two vectors approximates how similar their meanings are — independent of exact wording. MeetingMind uses this single capability in five distinct ways:

Use	How
1. Few-shot classification	Every extractor (actions, decisions, risks, questions) keeps a handful of hand-written positive and negative example sentences. For a candidate sentence, *margin* = max similarity to positives – max similarity to negatives. A weak rule match needs a positive margin to be kept; a strong rule match is only rejected if the margin is strongly negative. This is a training-free, few-shot “nearest prototype” classifier layered on top of the grammar rules.
2. De-duplication	Two candidate action items or decisions with cosine similarity ≥ 0.86 are treated as the same item restated and merged, instead of appearing twice.
3. Topic segmentation	TextTiling (Hearst, 1997) reimplemented with embeddings instead of word-overlap counts: at every gap between sentences, the mean embedding of the 3 sentences before is compared (cosine) with the mean embedding of the 3 sentences after. The resulting similarity curve's local dips (relative to nearby peaks) mark topic boundaries.
4. Keyphrase re-ranking	A KeyBERT-style score ranks YAKE's candidate phrases: 0.5 × (embedding similarity of the phrase to the whole meeting) + 0.3 × (YAKE rank) + 0.2 × (frequency), then Maximal Marginal Relevance (λ = 0.55) selects a diverse top set instead of near-duplicate phrases.
5. Summary grounding & evidence	Every generated summary sentence's cosine similarity against every transcript sentence is computed; the closest matches become its “supported by” evidence, and a sentence whose best similarity falls below 0.25 is removed outright as unsupported (see Section 8).

Fallback: if the embedding model cannot be loaded (download failure, disk space, …), a deterministic hashed bag-of-n-grams vector stands in for it. This keeps de-duplication and topic segmentation working in a degraded form, but the semantic veto in the classifier is switched off and the pipeline falls back to rules only, with a warning surfaced in the report.

7. Extraction Pipeline, Category by Category

Preprocessing

- Unicode NFKC normalisation, curly-quote straightening, invisible-character removal.
- A speaker-grammar regular expression recognises `Name: text`, timestamped variants, Otter/Teams-style headers and WebVTT `<v Name>` tags; a label only counts as a speaker if it looks like a name (≤4 capitalised words, not a stop-label like “Agenda”).
- `Meeting:`, `Date:` and `Attendees:` header lines are parsed out of the speech and used to resolve the meeting date and names.
- Consecutive lines from the same speaker merge into one utterance, then spaCy splits sentences; every sentence gets an id (`s12`) that all downstream evidence references.

Action items

Six candidate patterns, from most to least explicit: explicit `Action:` labels, self-commitments (“I'll…”), named third-person commitments (“Marcus will…”), requests (“Priya, can you…?”), explicit assignments, and bare imperatives — with long sentences split into clauses so mid-sentence commitments aren't missed.

Filtered out: past-tense reports, hypotheticals, negations, “no promises” offers, contingencies (treated as decisions instead), suggestions, and meeting logistics.

Owner resolution cascades: the speaker (for “I”) → the named person → the person addressed → the next speaker whose reply accepts (“Sure”, “Will do”) — marked as *inferred* → otherwise **Unassigned**, with a human-readable reason. Vague replies and follow-up requests are merged into the task they refer to via discourse tracking.

Deadlines

Explicit dates (“March 10”, “2026-04-01”) go through **dateparser**, relative to the meeting date. Relative expressions (today, tomorrow, weekdays, end of week/month/quarter, “in 3 days”, …) use a deterministic rule engine, not an LLM. Ambiguous cases — “next Friday” (this Friday or the one after?), a bare weekday equal to the meeting day, week/month-level deadlines — are flagged with an explanation rather than silently guessed. Vague timing (“ASAP”, “end of sprint”) is kept with **no date**: MeetingMind never invents a calendar date.

Decisions, risks & questions

- **Decisions** use separate cue sets for confirmed (“we decided”, “approved”), rejected (“decided against”, “out of scope”) and tentative/conditional (“for now”, “unless”) decisions, with suggestion/hypothetical/question filters. A tag question confirmed by another speaker, or a proposal followed by “Agreed”, becomes an *inferred* agreement.
- **Risks & blockers** use a problem/dependency lexicon (“blocked until”, “waiting on”, “worried”, “rate limited”) with filters for negations, commitments and recaps. Severity is a keyword heuristic (“critical”, “launch”, “security” → high), labelled as a heuristic in the UI.
- **Questions** count as unresolved if no other speaker gives a substantive answer in the next three turns, or if the reply defers it (“not sure yet”, “let's decide that next week”). A topic change counts as no answer.

Topic segmentation

TextTiling with embeddings (see Section 6, use 3). Agenda transitions (“Moving on to…”) add a bonus to the boundary score. Each topic is labelled with its best keyphrase, and its key points are the most central sentences under **TextRank** (PageRank over a sentence-similarity graph).

Keywords

YAKE (Campos et al., 2020) proposes n-gram candidates from unsupervised statistical features (casing, position, frequency dispersion) — no training data needed. Candidates are kept only if they sit inside a spaCy noun chunk. They are then re-ranked and diversified with embeddings and MMR (Section 6, use 4).

Summarization

Hierarchical (map-reduce) summarization handles transcripts far beyond DistilBART's 1024-token window:

- each topic's dialogue is split into chunks of at most 850 tokens, on speaker-turn boundaries;
- each chunk is summarized independently with 4-beam search and a 3-gram repeat penalty;
- if the combined chunk summaries are still long, they are summarized again;
- the executive summary is generated from the topic summaries.

Grounding checks (hallucination control) run on every generated sentence:

- **Removed** if it contains a capitalised name or number absent from the transcript;
- **Removed** if its best cosine similarity to any transcript sentence is below 0.25;
- **Flagged "weakly supported"** if two of its names/numbers never occur within one sentence of each other in the transcript (catches conflations like "Android 9 was moved to March 17");
- **Flagged "weakly supported"** if it claims a person *will* do something with no matching extracted action item owned by that person.

Each kept sentence carries its two most similar transcript sentences as evidence. Decision and follow-up summaries are generated *deterministically* from the already-extracted structured items, so that part cannot hallucinate at all. When the summarizer itself is unavailable, an extractive TextRank summary (boosted for decision/risk sentences) is used instead.

Recommended next steps

Rule-based, derived from the already-extracted items (not generated by a model): resolve blockers, assign missing owners/deadlines, confirm ambiguous dates and tentative decisions, mitigate high-severity risks, answer open questions, flag workload imbalance, prioritise the earliest deadline, schedule a follow-up. Each recommendation carries a rationale and links back to the evidence of the item it is based on, and the UI labels these as MeetingMind's suggestions, not statements from the meeting.

Reliability principles

- **Evidence everywhere:** every action item, decision, risk, question, entity, deadline and summary sentence references transcript sentence ids.
- **Explicit vs. inferred, tracked per field:** `owner_source` (named in transcript / self-commitment / addressed request / inferred from reply / unassigned / set by user), `date_type` (explicit / relative / vague / user), `grounding` (supported / weak).
- **No calibrated-probability claims:** the "strong / moderate / weak signal" labels shown in the UI are explicitly documented as heuristics, not statistics.
- **Never invent:** no owner without a traceable source, no date for vague timing, no decision from a suggestion without an explicit agreement.

8. Evaluation

Measured on 10 manually labelled transcripts (125 gold items total) across three splits: **dev** (used to tune the rules), **test** (first run frozen, then later revised — so its numbers are optimistic), and **holdout-2** (three transcripts written only after every rule change was finished, evaluated exactly once). Items are matched to gold labels by evidence sentence; a simple keyword-cue baseline is evaluated the same way for comparison.

Split	Actions F1	Decisions F1	Risks F1	Questions F1	Micro-F1	Baseline
Dev	0.93	0.93	0.80	0.80	0.88	0.65
Test (post-hoc)	0.97	1.00	0.89	1.00	0.96	0.61
Holdout-2 (unseen, run once)	0.86	0.67	0.53	1.00	0.76	0.53

*The held-out split is the fair number: **0.76 micro-F1** vs **0.53** for the baseline. The test split's score is optimistic because its extraction rules were adjusted after an error analysis of that exact split.*

Accuracy on matched items (holdout-2)

Attribute	System	Baseline
Owner accuracy	1.00 (12/12)	0.58 (7/12)
Deadline accuracy	1.00 (12/12)	0.42 (5/12)
Status accuracy	1.00 (4/4)	0.67 (2/3)
Kind accuracy	0.75 (3/4)	0.50 (1/2)

Where it still misses things

On unseen transcripts, decision and risk *recall* are the weakest spots (0.50 and 0.40 on holdout-2) — unusual phrasings fall outside the cue lexicons, e.g. “heavy rain Thursday afternoon”, “without harnesses”, or “keeping the tower crane until the roof is done” being missed as risks/decisions. Precision stays high throughout because the rules require explicit evidence, so missing an item (which a user can add manually) is preferred over inventing one.

Testing

112 backend tests (pytest) covering extraction rules, deadline resolution, file extraction, models, the API surface and longer integration cases, plus **15 frontend tests** (Vitest + Testing Library) covering every report section and the action-item editing flow. All pass; TypeScript strict-mode typecheck and the production build are clean.

9. Frontend & User Experience

The React SPA presents the same report through **12 views** — overview, executive summary, discussion topics, decisions, action items, deadlines, risks & blockers, unresolved questions, people & entities, keywords, transcript evidence (every tagged sentence, clickable from anywhere else in the report), and recommended next steps.

All state (the draft transcript, the current report, and any edits) lives only in memory for that browser tab; nothing is written to local storage except the theme preference and nothing reaches a database.

Visual design

Two themes, a single token system: a dark “Luminous Obsidian” theme (near-black surfaces, violet/lime accents, glass cards, JetBrains Mono labels) and a light “Morning Lavender” theme (warm white surfaces, the same violet primary, a hot-pink “live” accent), switchable from a sun/moon toggle that persists the choice and applies it before first paint to avoid a flash. Typography: Plus Jakarta Sans for headings, Inter for body text, JetBrains Mono for labels/timestamps/technical values.

Exports

PDF (this document's own renderer — ReportLab, with a Unicode TTF when available), Markdown, CSV (formula-injection safe, UTF-8 BOM for Excel) and JSON, all generated server-side on demand and always reflecting the user's latest edits.

10. Privacy

- Models run on the machine that starts the MeetingMind server — no paid or third-party AI APIs are called anywhere in the pipeline.
- No database, no accounts, no meeting history. Uploaded files are deleted immediately after text extraction.
- Transcript text is never written to logs.
- Reports exist only in the browser tab; they must be exported before starting a new analysis or they are discarded.

Limitation acknowledged in the project itself: *whoever runs the backend could modify it to log data, and the first run downloads model weights from Hugging Face (never transcript text).*

11. Limitations & Future Work

Known limitations

- Decision/risk recall on unseen transcripts is the weakest area (0.50 / 0.40 on holdout-2) — unusual phrasings fall outside the cue lexicons.
- The evaluation set is small (10 transcripts) and was labelled by one person, with no inter-annotator agreement.
- Abstractive summaries can still merge or misattribute details that the grounding checks don't catch (e.g. two people mentioned in adjacent lines).
- The small spaCy model mislabels some organisation/product entities; pronoun resolution (“he will…”) is not attempted.

- Without `Name:` speaker labels, self-commitments can't get an owner.
- CPU analysis of a ~650-word transcript takes roughly 25–40 seconds, mostly summarization; one analysis runs at a time, others queue.
- English only; no OCR for scanned PDFs; no audio/speech-to-text.

Planned enhancements

- Train a lightweight classifier (logistic regression or a fine-tuned MiniLM) on a larger labelled corpus (e.g. AMI/ICSI) to raise decision and risk recall.
- Coreference resolution for pronoun owners; an optional transformer NER model.
- Speech-to-text (e.g. Whisper) for audio uploads, with speaker diarisation.
- Natural-language inference (NLI) checks for stronger summary-faithfulness verification.
- Calendar (.ics) export of deadlines and task-tracker integrations.
- Optional encrypted storage of reports for teams that want history.

12. NLP Techniques Checklist — What's Used, How & Why

Mapped against a standard NLP-course syllabus (preprocessing through RAG), to show exactly which techniques MeetingMind actually runs in production, which it uses in an adapted form, and which it deliberately doesn't need.

Technique	Used?	How it's used, and why
Preprocessing	Yes	Unicode normalisation, speaker/timestamp grammar, header parsing, utterance merging — the entry point for every other step.
POS tagging	Yes	spaCy POS tags detect imperatives (a bare root verb with no subject → a command, e.g. “Fix the bug.”), feeding the action-item rules.
Tokenization	Yes	spaCy's sentence and word tokenizer underlies every downstream step (NER, embeddings, YAKE).
Vectorization	Yes	Sentence embeddings (MiniLM, 384-dim) are the main vectorization; a hashed n-gram (bag-of-n-grams) vectorizer is the offline fallback.
N-grams	Yes	YAKE's keyphrase candidates are n-grams; the summarizer's beam search applies an n-gram repeat penalty; the fallback vectorizer hashes n-grams.
Named Entity Recognition	Yes	spaCy NER (PERSON, ORG, GPE, DATE, …), merged with speaker labels to build the People & Entities view.
Twitter-style text preprocessing	No	Not a relevant input format — transcripts are meeting dialogue, not tweets.
Web scraping	No	No content is ever fetched from the web. Transcripts are supplied directly by the user (paste or file upload) — scraping would work against the privacy goal of nothing leaving the machine.
Text classification pipeline	Adapted	No trained classifier / labelled training set. Instead, a **training-free few-shot “nearest-prototype”** classifier: hand-written positive/negative example sentences, and a candidate is accepted or vetoed by its embedding-similarity margin to them. Chosen because there was no large labelled meeting corpus to train on.
Topic modeling (LDA)	No (different technique)	Topics come from **TextTiling**: boundaries are placed where embedding similarity between neighbouring sentence windows dips. LDA models topics as word-distributions across a *corpus* of documents; MeetingMind needs to segment *one* transcript into a timeline of topics instead, which TextTiling is built for.
TextRank / TF–IDF cosine / BM25	Partial	**TextRank is used** — as the extractive summary fallback, and to pick each topic's key points — but scored with **embedding cosine similarity** instead of TF–IDF, which captures meaning rather than just shared words. **BM25 is not used**: there is no document-ranking/search step in the pipeline.
Text similarity	Yes — extensively	Cosine similarity between sentence embeddings is the single most-used technique in the whole pipeline: classification margins, de-duplication, topic boundaries, keyword re-ranking, and summary grounding (Section 6) all come down to it.
News-article analysis	No	Out of scope — the input domain is meeting dialogue, not news text.
RNN / LSTM / GRU / BiLSTM	No	All three models used are pretrained **Transformers**; no recurrent network is trained or run anywhere in the pipeline.

Technique	Used?	How it's used, and why
Sentiment analysis	No	MeetingMind doesn't score tone/sentiment anywhere in its actual output. (An earlier UI mockup explored a decorative “team sentiment” widget, but it was never wired to a real model or metric.)
Sentiment analysis via Transformers	No	Same as above — not part of the product.
Encoder-only model (e.g. DistilBERT)	Conceptually	The embedding model, MiniLM, *is* an encoder-only Transformer, distilled the same way DistilBERT is — but the specific model is `all-MiniLM-L6-v2`, not DistilBERT itself.
Decoder-only model	No	No GPT-style decoder-only generative model is used anywhere.
Encoder-decoder model	Yes — directly	The summarizer, **DistilBART**, is literally an encoder-decoder (seq2seq) architecture: a 12-layer encoder reads the dialogue chunk, a 6-layer decoder generates the summary.
RAG (vector DB / FAISS / ChromaDB)	No	There is no retrieval step and no persistent vector database. Embeddings are computed fresh per analysis, held in memory, and compared only *within* that one transcript's own sentences — there is no external corpus being retrieved from, so this isn't retrieval-augmented generation even though both use embeddings.

Summary: 7 techniques are used directly as taught (preprocessing, POS tagging, tokenization, vectorization, n-grams, NER, text similarity); 3 are used in a deliberately adapted form for this problem (few-shot classification instead of a trained pipeline, TextRank scored by embeddings instead of TF–IDF/BM25, an encoder-only embedding model related to but not identical to DistilBERT); 1 is a direct architectural match (encoder-decoder = DistilBART); and web scraping, LDA, BM25, RNN/LSTM/GRU, sentiment analysis, decoder-only models and RAG/vector databases are not part of the product because the problem didn't call for them.

13. APIs Used, and Why

No paid or third-party AI APIs are called anywhere in MeetingMind. No OpenAI, no Anthropic, no Google Cloud NLP, and no Hugging Face *inference* API. This was a deliberate design constraint, not an oversight: the project had to run **offline, on a single laptop, for free**, and a tool whose outputs people act on should not depend on an external service's uptime, cost or data policy.

Interface	What it actually is	Why it's used
Hugging Face `transformers` (`AutoModelForSeq2SeqLM`, `AutoTokenizer`)	A **local** Python library API — downloads model weights once from the Hub, then runs entirely on the machine's own CPU/GPU, no further network calls.	Loads and runs DistilBART for summarization with no per-request network round-trip and no data leaving the machine.
`sentence-transformers` (`SentenceTransformer`)	Same — a local library API over the same kind of cached, locally-run model weights.	Loads and runs MiniLM to produce sentence embeddings on-device.
spaCy (`spacy.load`)	Local library API.	Loads `en_core_web_sm` for tokenisation, POS tagging, parsing and NER.
MeetingMind's own REST API (FastAPI)	The *only* network API in the whole system — and it's internal.	Lets the React frontend talk to the Python backend on the user's own machine: `/api/extract`, `/api/analyze/stream`, `/api/export/<format>`, `/api/health`. Not a third-party API — both ends run locally.

Net effect: **zero external AI API calls**, three pretrained models loaded through local library APIs, and one small internal REST API connecting the app's own frontend to its own backend. This is why MeetingMind can claim “nothing is stored, nothing is sent anywhere” as a real property of the system rather than a policy promise.

MeetingMind — your meetings, made meaningful. Generated from the project's own source code and documentation.